Module 07: Perimeter Defence Evasion Techniques

```
# Exercise 01: Scanning against Defenses with Nmap
```

Scenario

Perimeter defense mechanisms, such as stateful firewalls, access control lists (ACLs), and intrusion detection systems (IDS), actively inspect and filter incoming network traffic. When default security policies block or drop packets, standard active port scans fail or return ambiguous results.

To assess target attack surfaces effectively, penetration testers must evaluate how filtering devices behave under various configurations. By observing traffic patterns in protocol analyzers (such as Wireshark) while adjusting netfilter/iptables policies on target hosts—transitioning from open policies to explicit packet dropping (**DROP**) or ICMP prohibition responses (**REJECT --reject-with icmp-host-prohibited**)—testers can detect the presence of intermediary filters and formulate advanced evasion scanning strategies.

In this exercise, we initialize Wireshark on Parrot Security to capture traffic targeting an Ubuntu server (**192.168.0.51**), configure iptables policies on Ubuntu to drop incoming packets using the **DROP** target, observe scanning anomalies with Nmap (**nmap -sC 192.168.0.51 -n**), and analyze ICMP host-prohibited firewall responses in Wireshark.

Objective

The objective of this lab is to help students learn how to:
- Capture and analyze raw network protocol traffic using Wireshark on network interface **eth0**
- Perform initial target ping probes and active service discovery scans using Nmap (**nmap -sC 192.168.0.51 -n**)
- Configure Linux netfilter/iptables default input chain policies to silently discard packets using the **DROP** target
- Observe Nmap scanning behavior and packet timeout characteristics against drop-filtered targets
- Configure iptables ICMP host-prohibited rules (**-j REJECT --reject-with icmp-host-prohibited**) to emulate router ACL responses

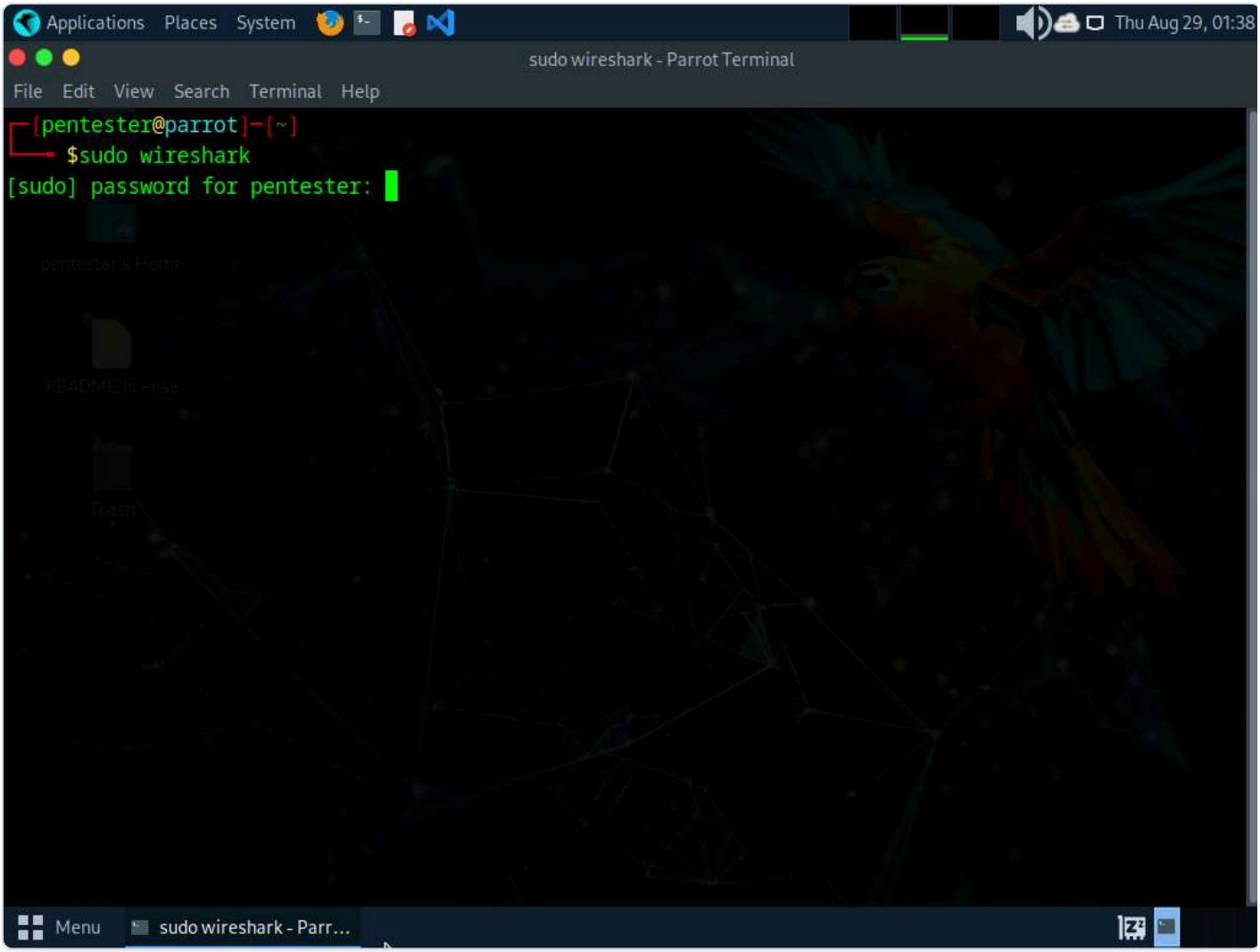
---

Task 1: Analyze Filtering Behavior with Wireshark and Nmap

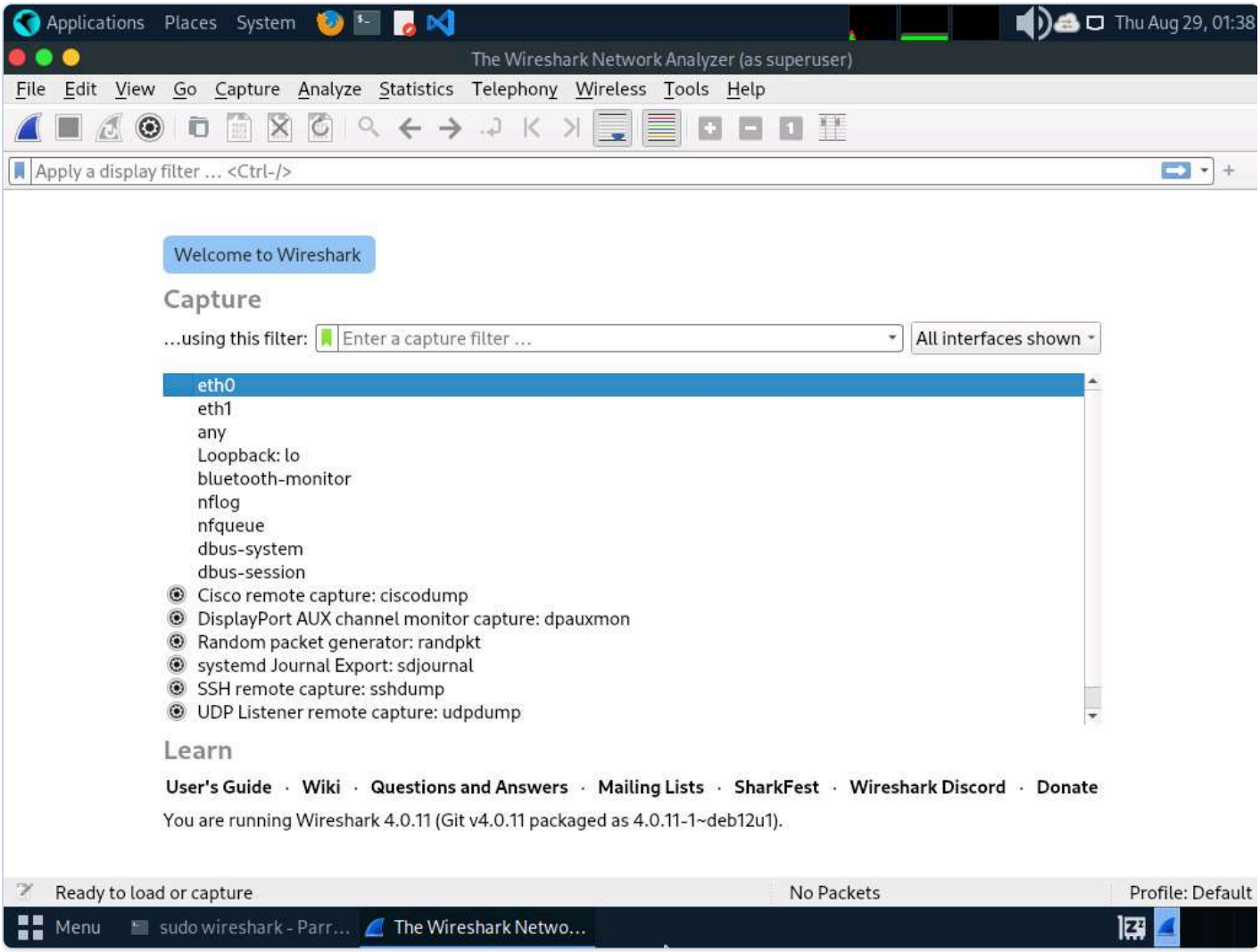
In this task, we capture ICMP and TCP traffic in Wireshark on Parrot Security, alter netfilter input policies on Ubuntu to enforce traffic drops using **DROP**, evaluate Nmap scan output against filtered targets, and emulate perimeter ACL responses using ICMP rejection rules.

Lab Tasks

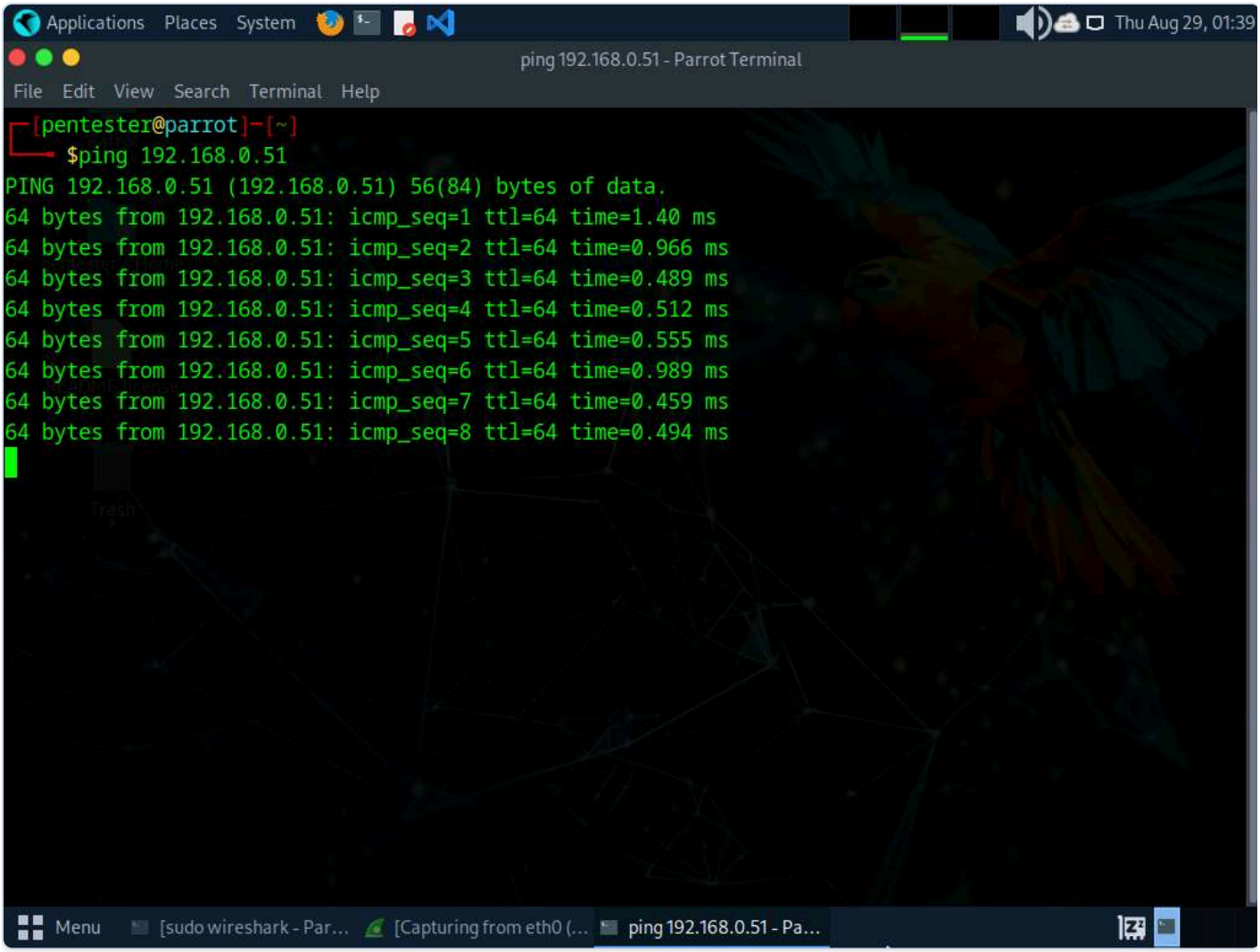
1. Switch to **Parrot Security** (**toor**). Open a terminal and launch Wireshark with elevated privileges:
   sudo wireshark



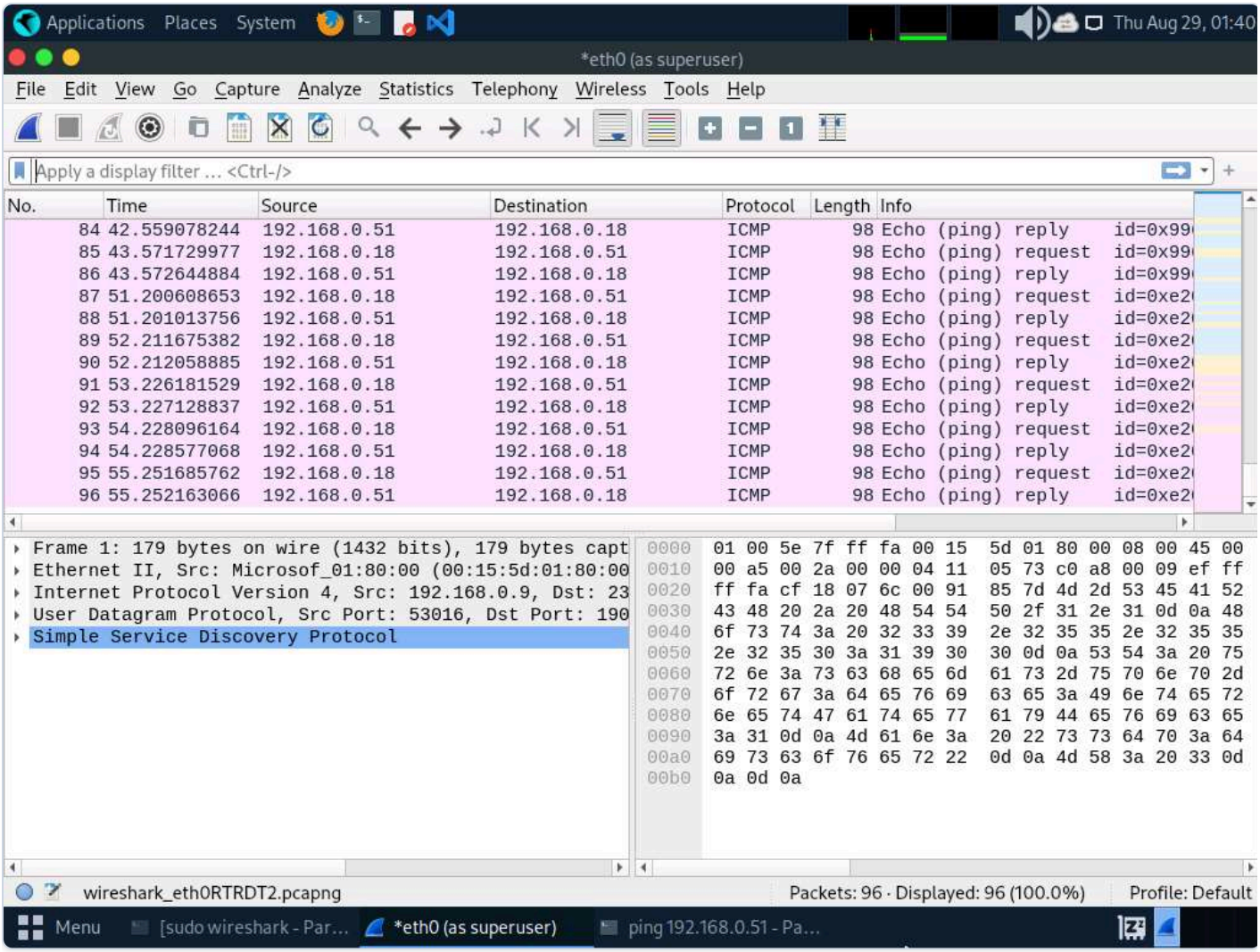
1. Double-click the **eth0** interface to start capturing network packets. Minimize the Wireshark window.



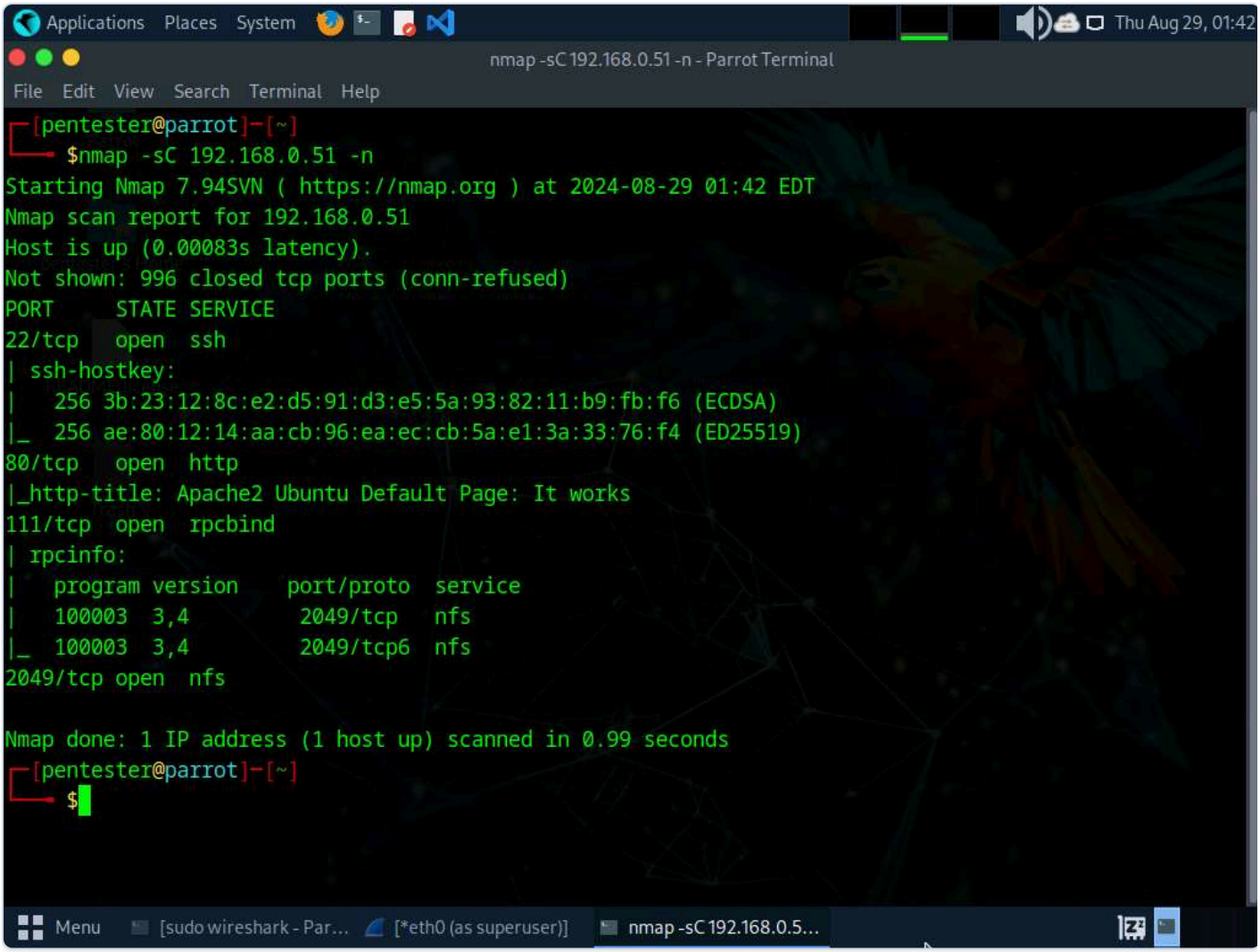
1. Open a new terminal window and verify connectivity to the Ubuntu target (192.168.0.51):
   ping 192.168.0.51



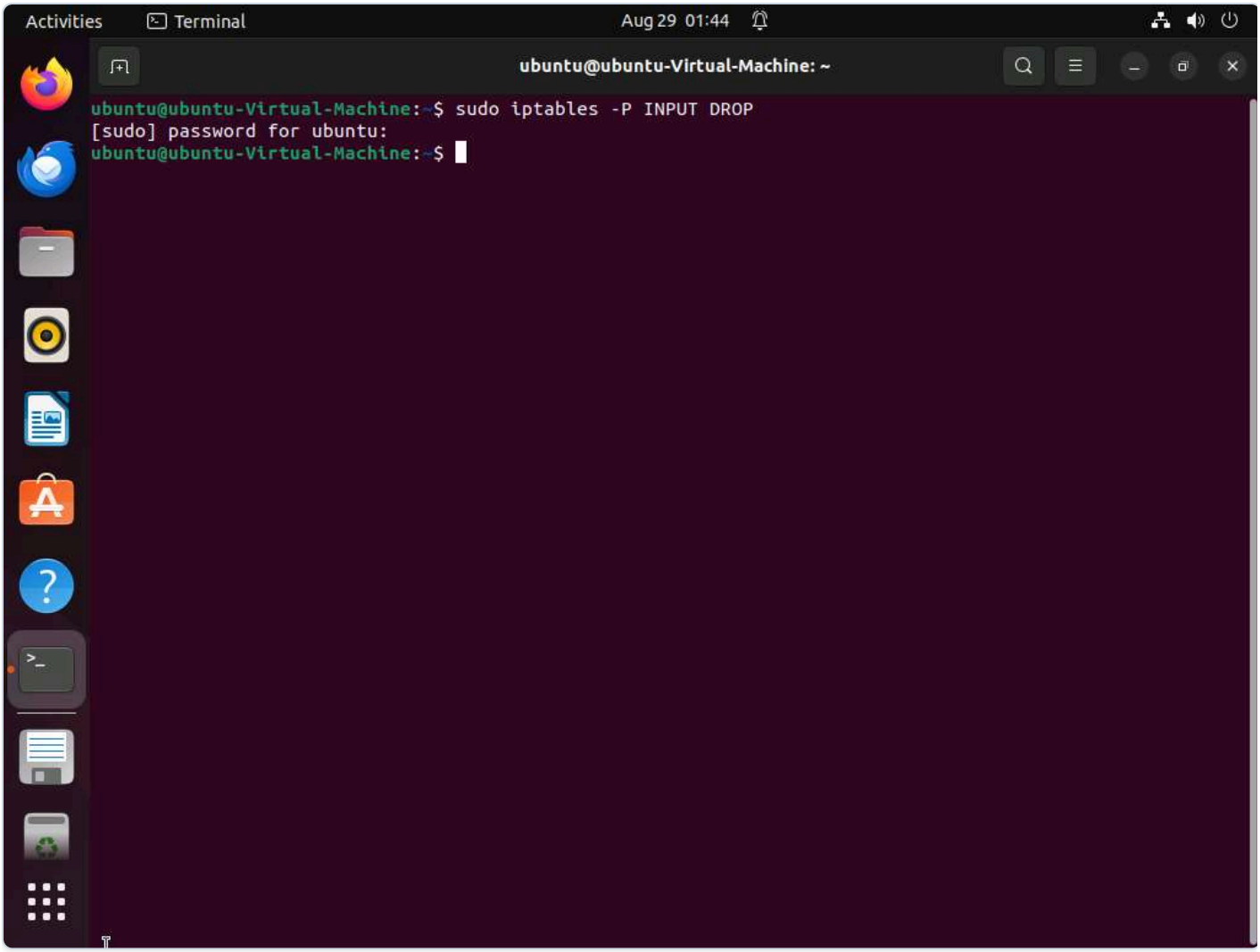
1. Switch back to Wireshark and click the **Stop Capturing Packets** button to review captured ICMP echo request/reply packets.



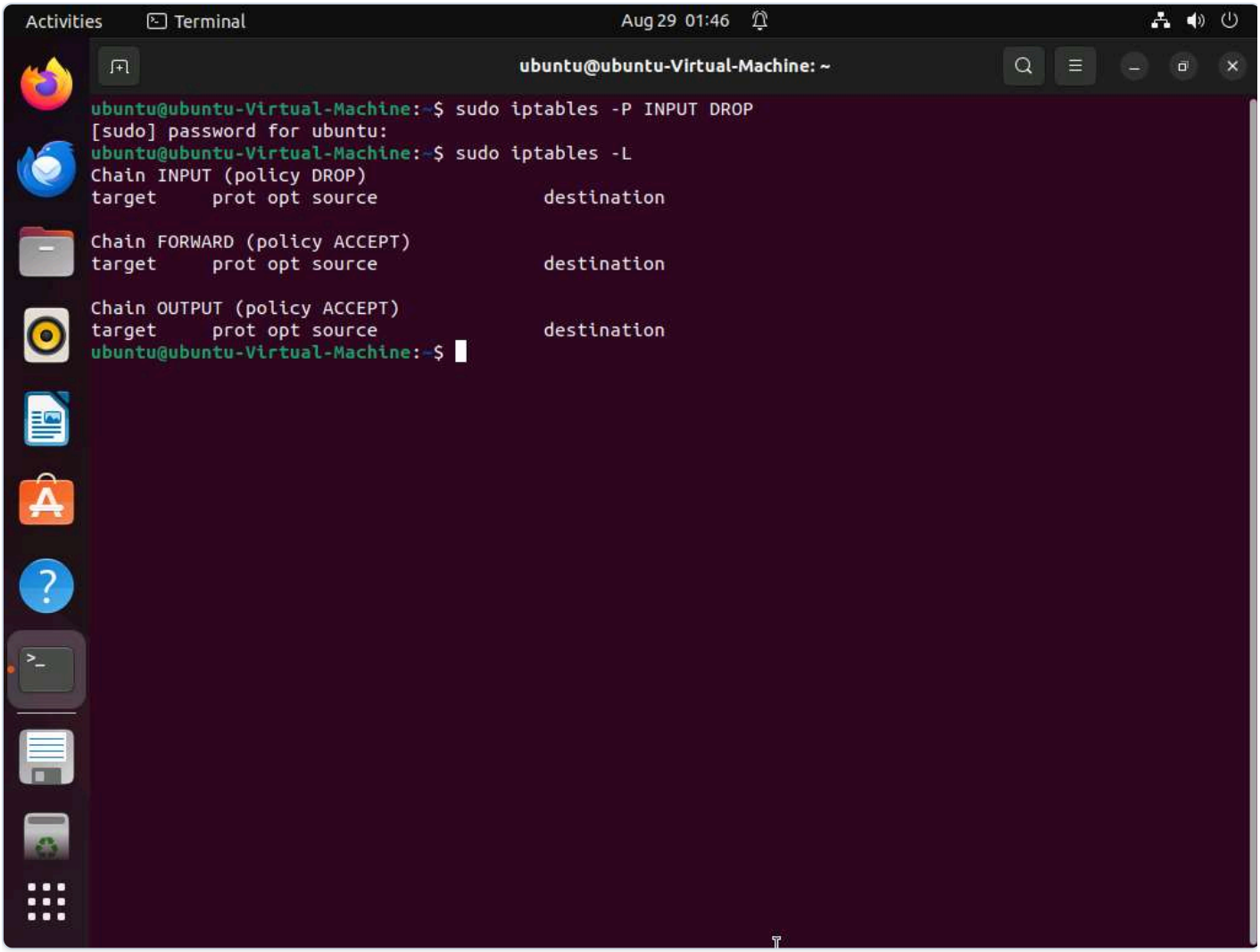
1. Open a terminal and execute an initial Nmap script scan against the target:
   nmap -sC 192.168.0.51 -n



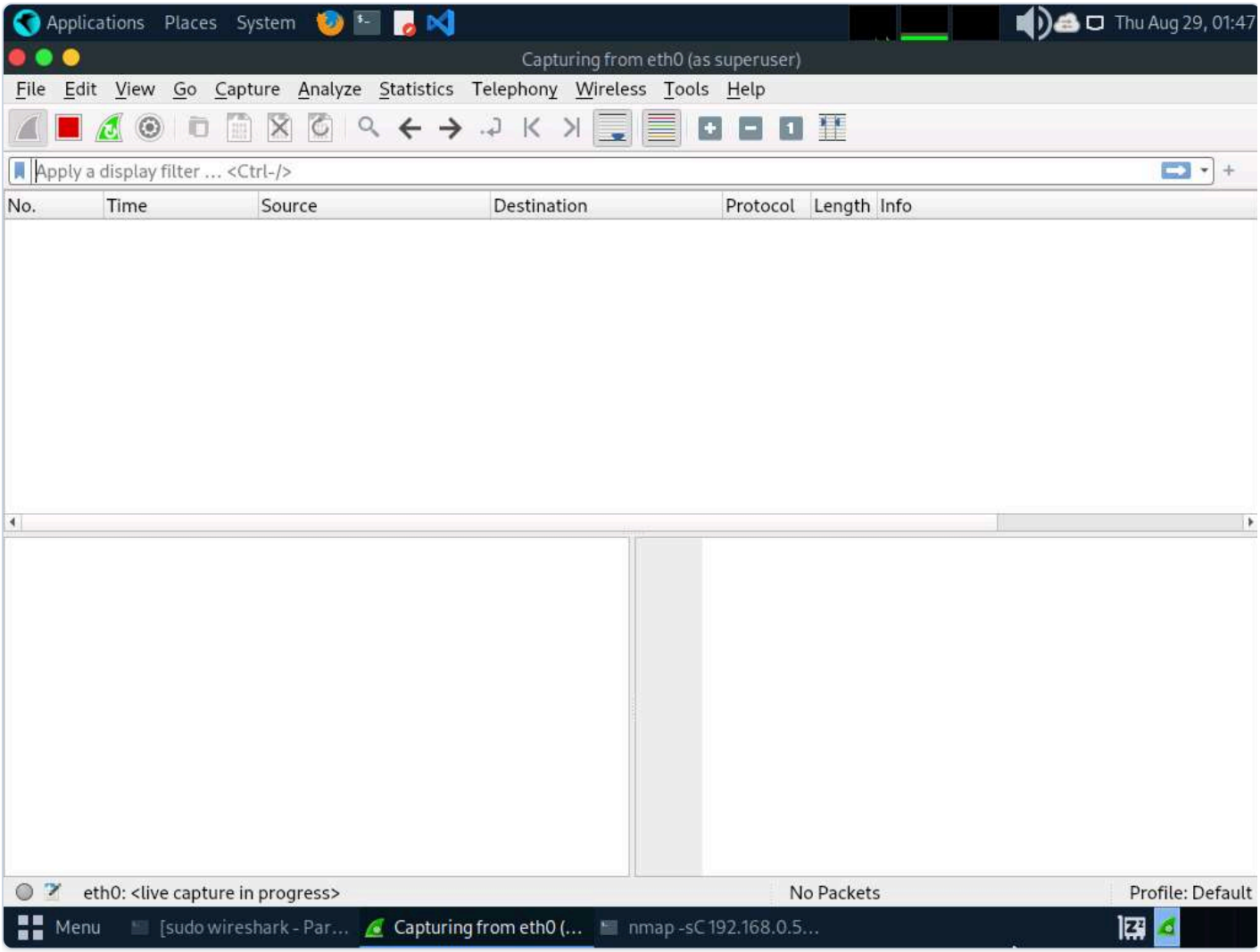
1. Switch to the **RPC Server Ubuntu** machine (**administrator** / **toor**). Launch a terminal elevated to root and set the default INPUT chain policy to silently discard all incoming traffic using the **DROP** target:
   sudo iptables -P INPUT DROP



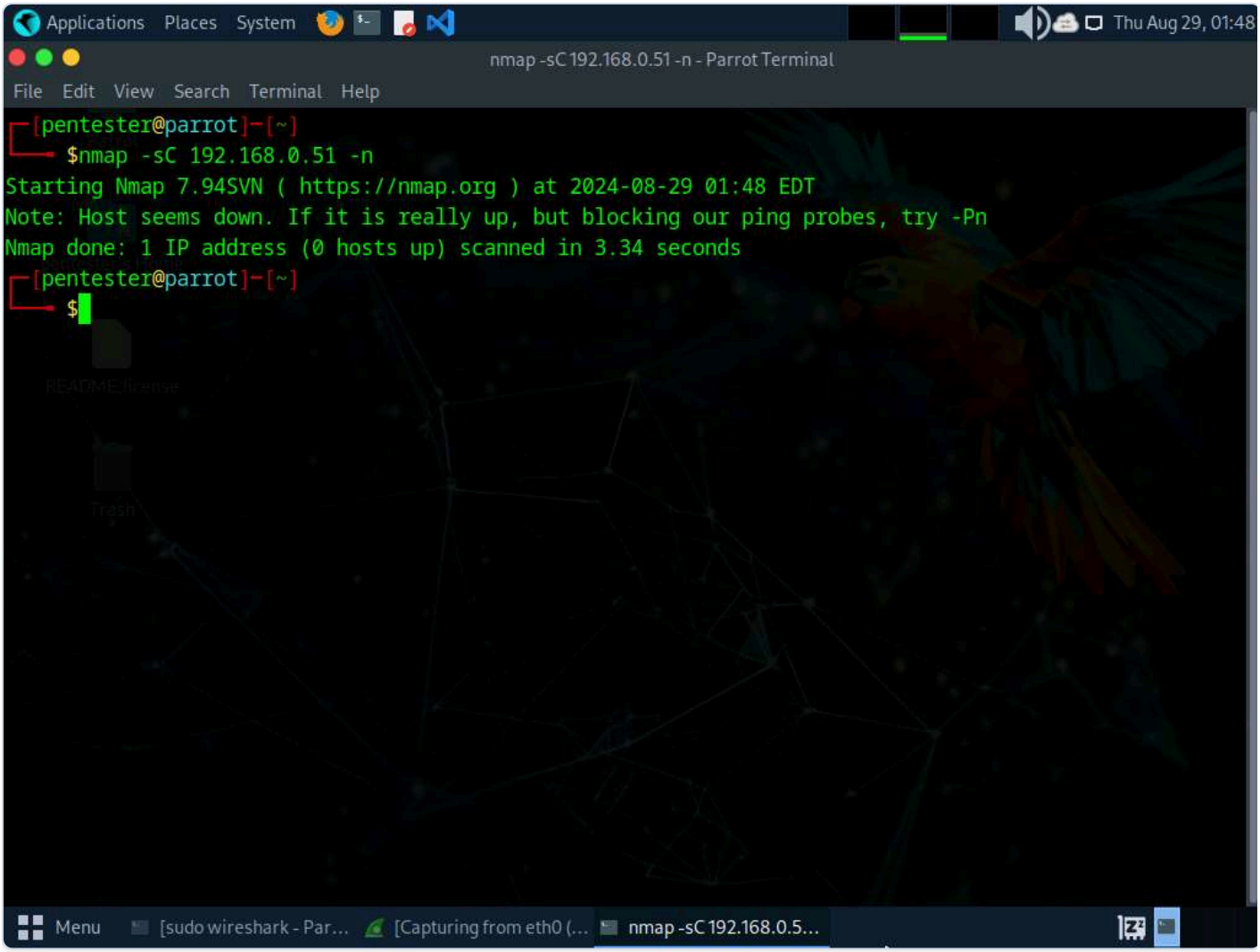
1. Inspect the active iptables rule structure:
   sudo iptables -L



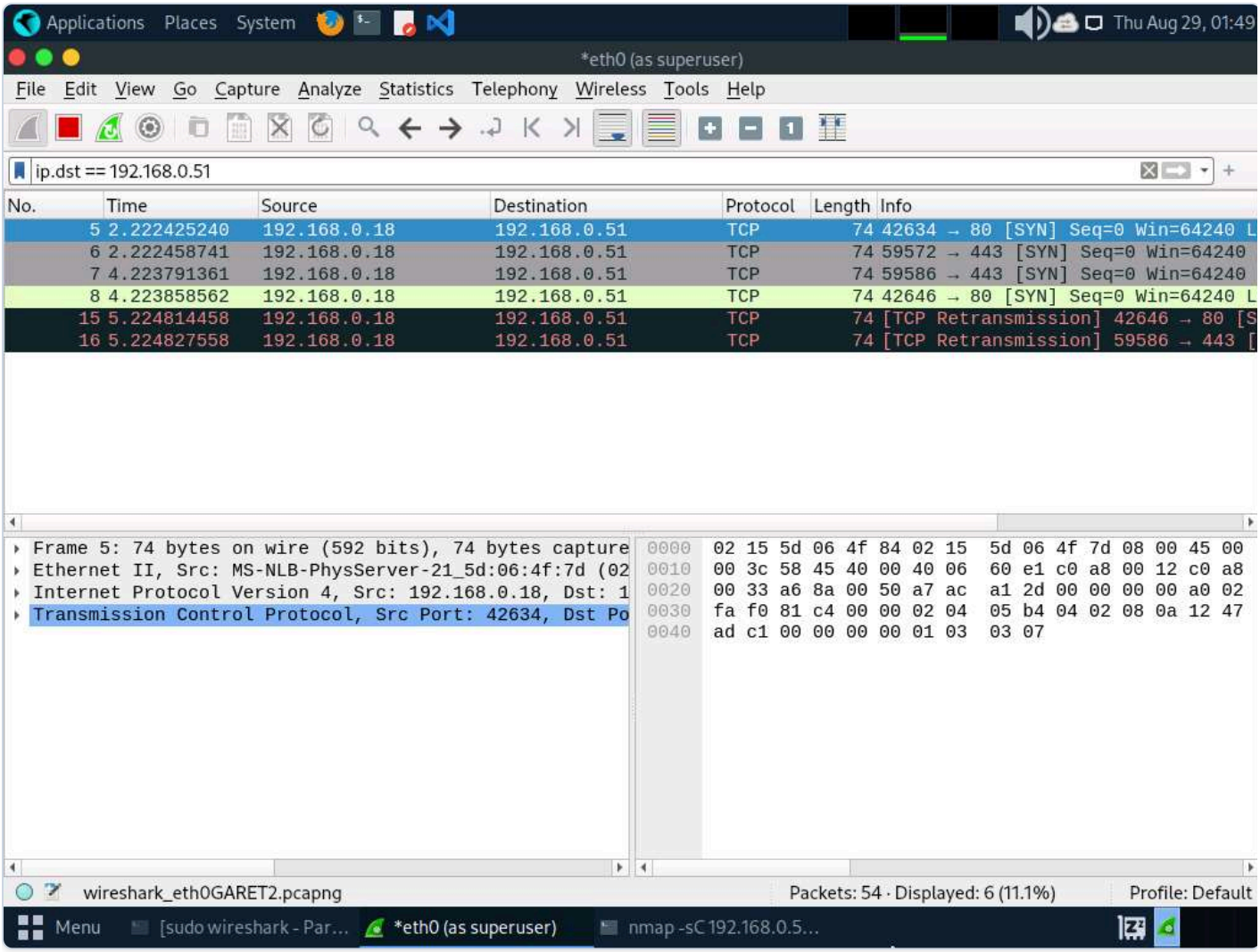
1. Switch to Parrot Security, maximize Wireshark, and click **Start capturing packets** (Shark Fin Icon). Select **Continue without Saving** if prompted.



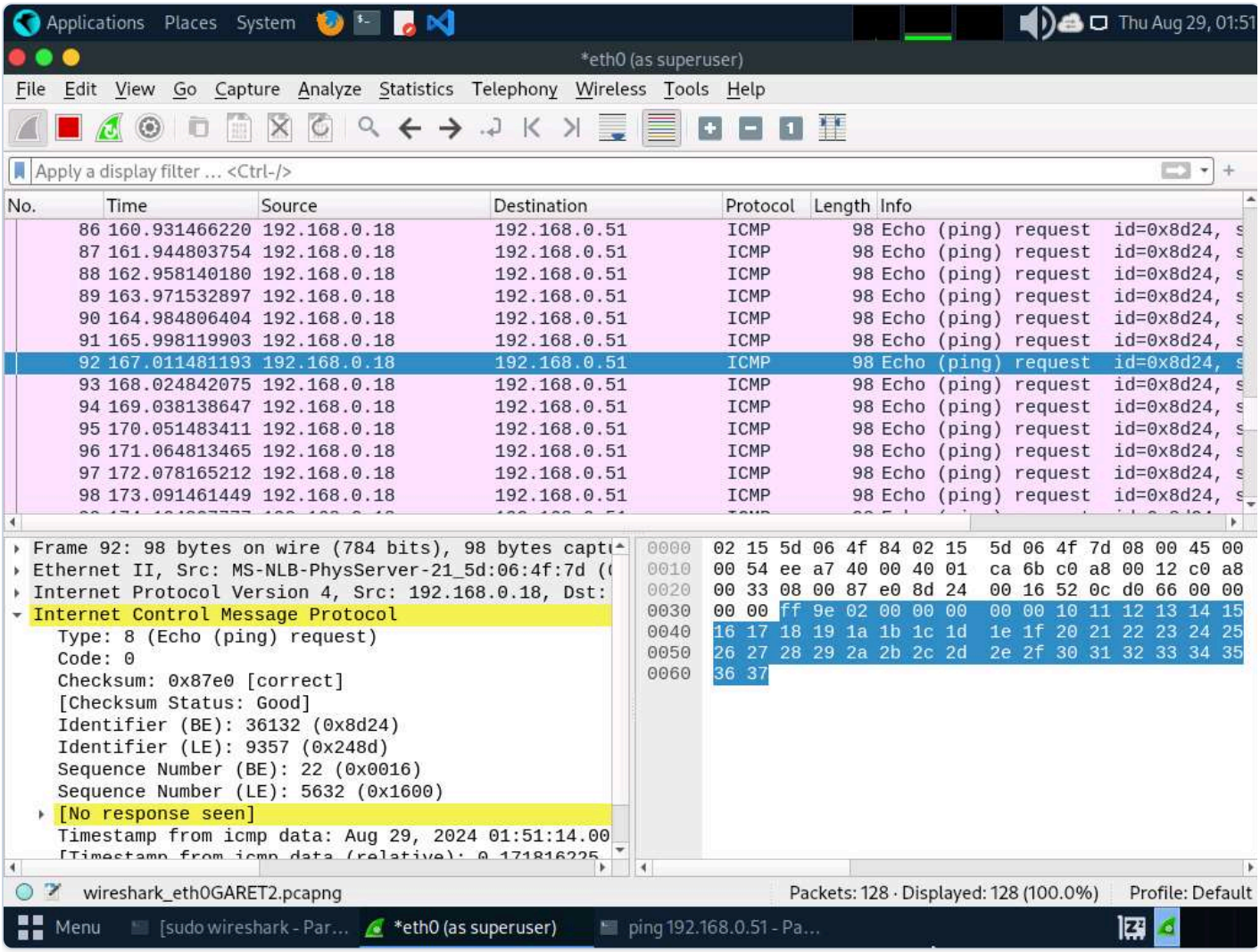
1. Open a terminal on Parrot Security and re-run the Nmap scan against the drop-filtered target:
   nmap -sC 192.168.0.51 -n



1. In Wireshark, apply the destination filter to isolate target traffic:
   ip.dst == 192.168.0.51



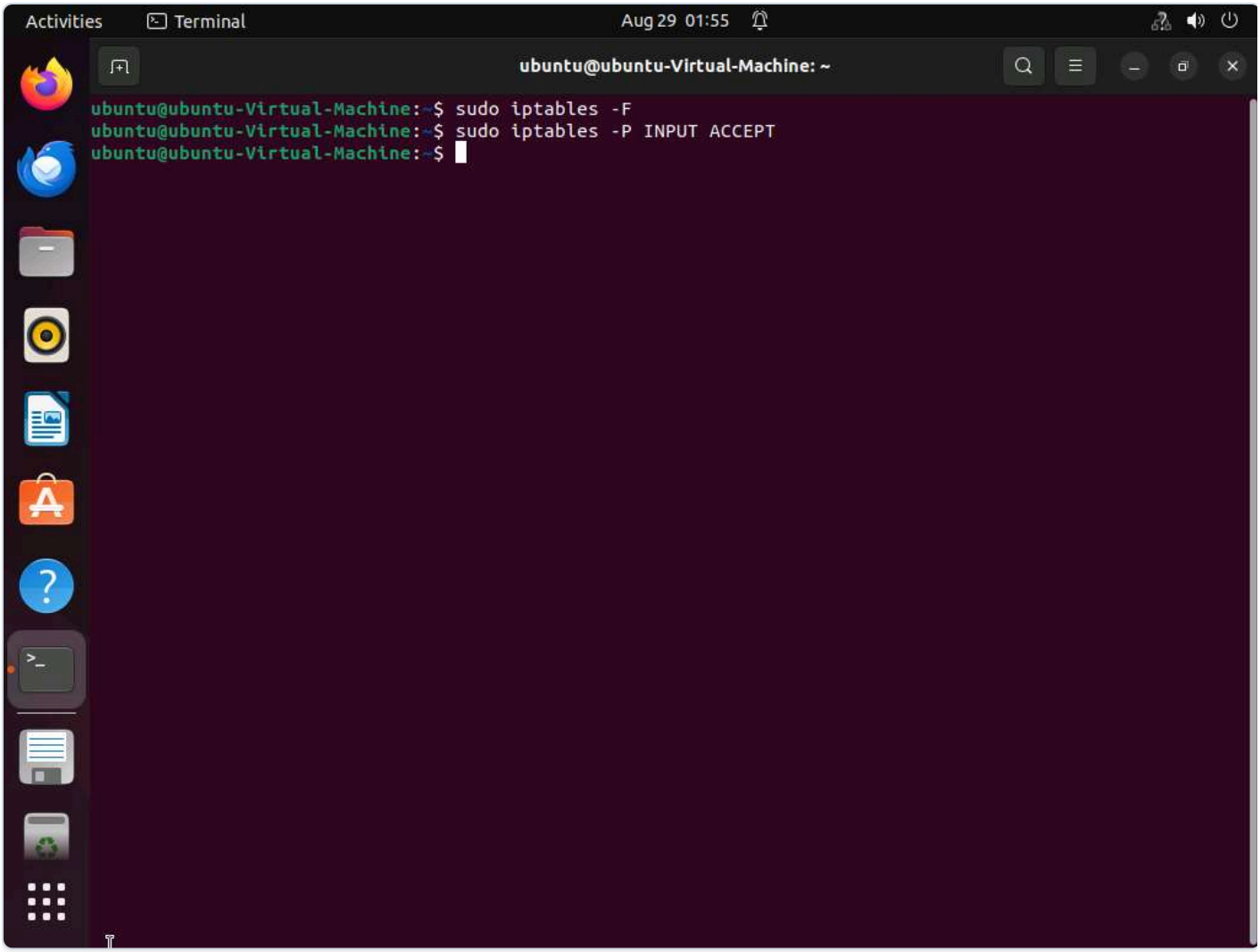
1. Open a terminal and attempt to ping the target again (**ping 192.168.0.51**). Observe in Wireshark that no ICMP responses are returned due to the **DROP** policy.



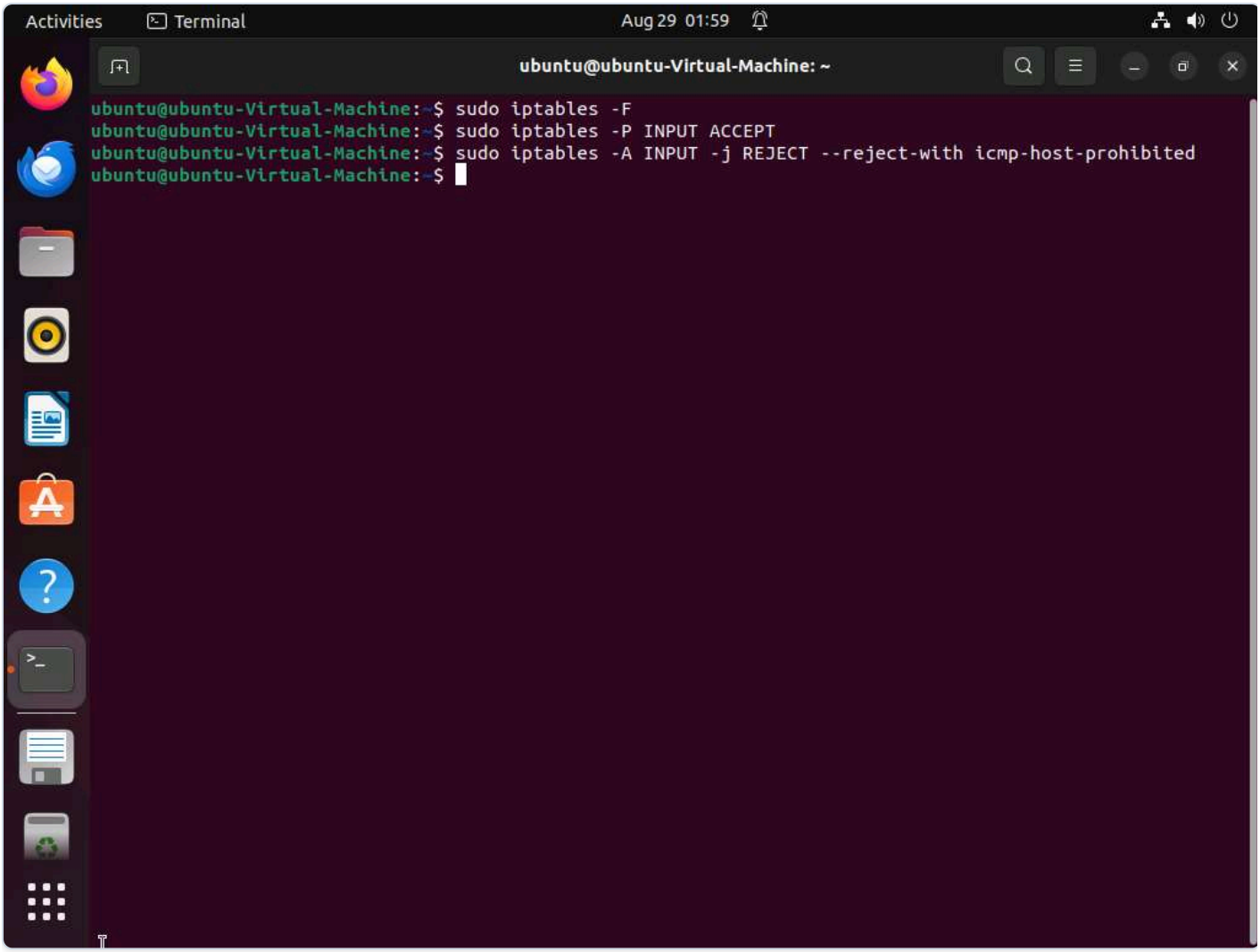
1. Switch to the **RPC Server Ubuntu** machine to re-configure iptables to emulate typical router access control list (ACL) ICMP responses. Reset existing chain policies:

   sudo iptables -F

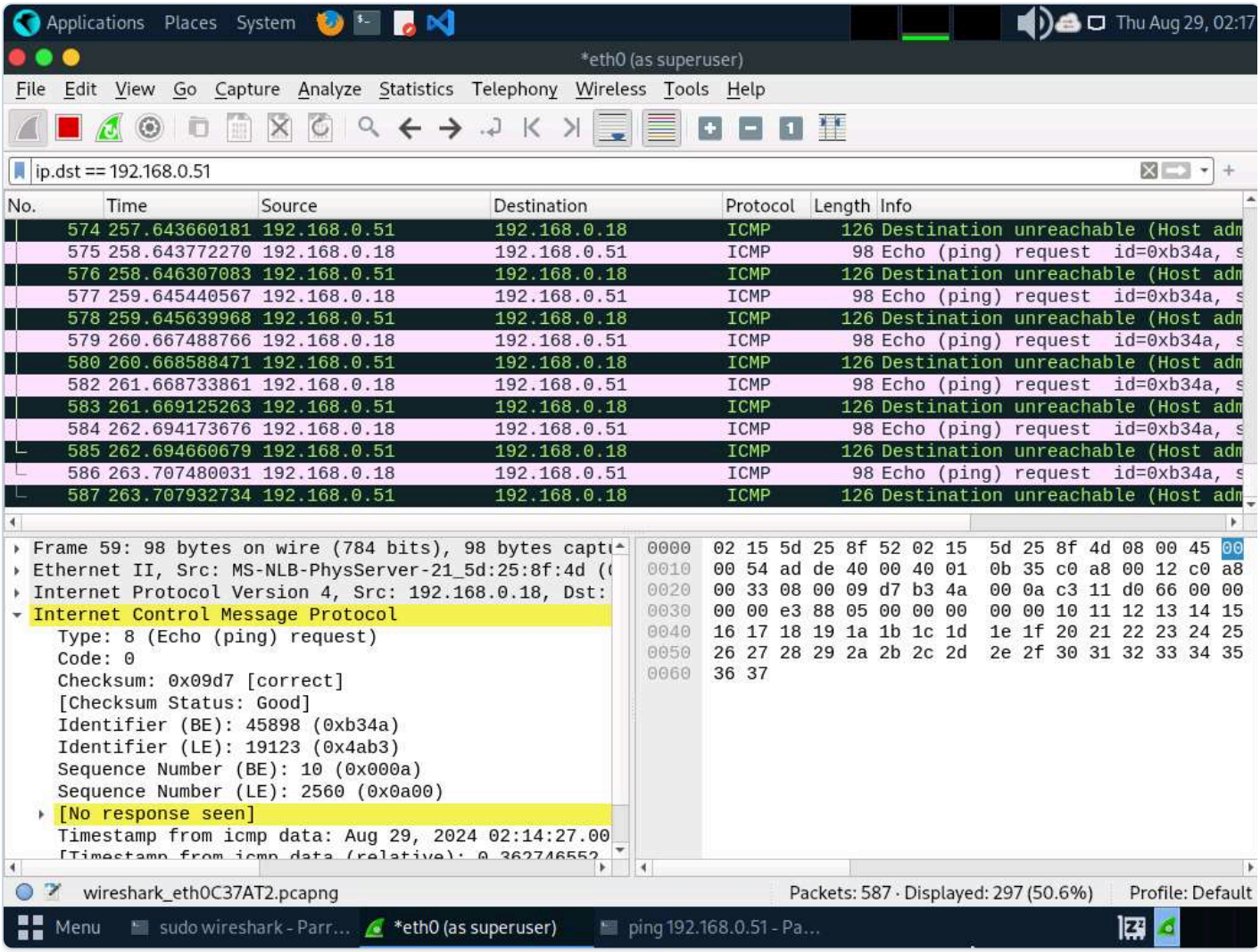
   sudo iptables -P INPUT ACCEPT

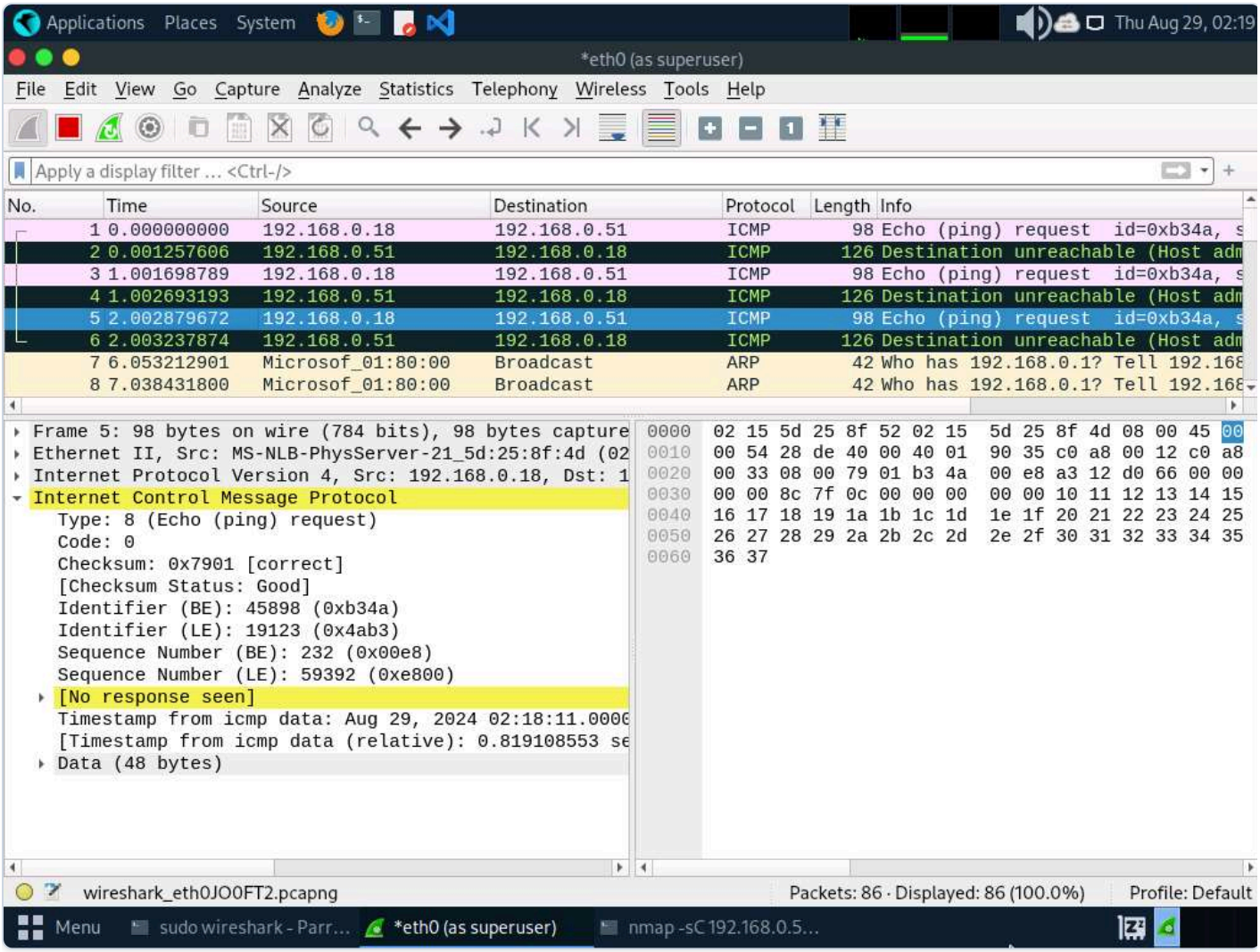


1. Append an explicit rejection rule that returns ICMP host-prohibited packets (Type 3, Code 13):
   sudo iptables -A INPUT -j REJECT --reject-with icmp-host-prohibited



1. Switch to Parrot Security, restart packet capture in Wireshark, and execute the Nmap scan:
   nmap -sC 192.168.0.51 -n


Applications Places System
Thu Aug 29, 02:17
*eth0 (as superuser)
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
ip.dst == 192.168.0.51
No. Time Source Destination Protocol Length Info
574 257.643660181 192.168.0.51 192.168.0.18 ICMP 126 Destination unreachable (Host adm
575 258.643772270 192.168.0.18 192.168.0.51 ICMP 98 Echo (ping) request id=0xb34a, s
576 258.646307083 192.168.0.51 192.168.0.18 ICMP 126 Destination unreachable (Host adm
577 259.645440567 192.168.0.18 192.168.0.51 ICMP 98 Echo (ping) request id=0xb34a, s
578 259.645639968 192.168.0.51 192.168.0.18 ICMP 126 Destination unreachable (Host adm
579 260.667488766 192.168.0.18 192.168.0.51 ICMP 98 Echo (ping) request id=0xb34a, s
580 260.668588471 192.168.0.51 192.168.0.18 ICMP 126 Destination unreachable (Host adm
582 261.668733861 192.168.0.18 192.168.0.51 ICMP 98 Echo (ping) request id=0xb34a, s
583 261.669125263 192.168.0.51 192.168.0.18 ICMP 126 Destination unreachable (Host adm
584 262.694173676 192.168.0.18 192.168.0.51 ICMP 98 Echo (ping) request id=0xb34a, s
585 262.694660679 192.168.0.51 192.168.0.18 ICMP 126 Destination unreachable (Host adm
586 263.707480031 192.168.0.18 192.168.0.51 ICMP 98 Echo (ping) request id=0xb34a, s
587 263.707932734 192.168.0.51 192.168.0.18 ICMP 126 Destination unreachable (Host adm
Frame 59: 98 bytes on wire (784 bits), 98 bytes capt
Ethernet II, Src: MS-NLB-PhysServer-21_5d:25:8f:4d (
Internet Protocol Version 4, Src: 192.168.0.18, Dst:
Internet Control Message Protocol
Type: 8 (Echo (ping) request)
Code: 0
Checksum: 0x09d7 [correct]
[Checksum Status: Good]
Identifier (BE): 45898 (0xb34a)
Identifier (LE): 19123 (0x4ab3)
Sequence Number (BE): 10 (0x000a)
Sequence Number (LE): 2560 (0x0a00)
[No response seen]
Timestamp from icmp data: Aug 29, 2024 02:14:27.00
0000 02 15 5d 25 8f 52 02 15 5d 25 8f 4d 08 00 45 00
0010 00 54 ad de 40 00 40 01 0b 35 c0 a8 00 12 c0 a8
0020 00 33 08 00 09 d7 b3 4a 00 0a c3 11 d0 66 00 00
0030 00 00 e3 88 05 00 00 00 00 00 10 11 12 13 14 15
0040 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23 24 25
0050 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 34 35
0060 36 37
wireshark_eth0C37AT2.pcapng
Packets: 587 · Displayed: 297 (50.6%)
Profile: Default
Menu
sudo wireshark - Parr...
*eth0 (as superuser)
ping 192.168.0.51 - Pa...



1. Before ending the lab, restore the open input policy on the Ubuntu machine:
   sudo iptables -P INPUT ACCEPT

2. Close all active terminal and Wireshark windows.

This concludes the lab exercise.

IMPORTANT: ❓ **Question 7.1.1.1** Using the Parrot Security machine, conduct a scan on the target system, Ubuntu (192.168.0.51), utilizing advanced scanning options to bypass filtering mechanisms. Configure iptables on the Ubuntu machine to restrict incoming traffic. Specify the iptables target used in this lab to enforce traffic blocking on the Ubuntu system. [!SUCCESS] ✅ **Answer:** **DROP**